

What is Sequence Modeling ?(A sequence is simply an ordered collection of things)

Suppose you have a normal image. cnn looks at the image and predicts this is a cat. The image is treated as one input. But now consider a video : Frame 1 → Frame 2 → Frame 3 → Frame 4 → Frame 5 here each frame is an image. If we process every frame independently we don't understand what is happening over time . The meaning of Frame 5 becomes much clearer when we know what happened in Frames 1–4.

Text : I → love → machine → learning

Video : Frame1 → Frame2 → Frame3 → Frame4

Audio : sample1 → sample2 → sample3 → ...

Order matters. Sequence modeling can happen across **time or space** .

Even an image can be converted into an ordered sequence: Patch1 → Patch2 → Patch3 → Patch4 → ... used by vision transformers

9. Vision Transformers and spatial sequences

Your PPT also mentions spatial sequence modeling.

Instead of viewing an image as:

ONE BIG IMAGE

we can divide it into patches:

P1	P2	P3	P4
P5	P6	P7	P8

Then:

P1 → P2 → P3 → ... → P8

becomes a sequence.

But there is a problem.

The model needs to know:

Where did this patch come from?

That's why Transformers use **positional encoding**.

Why can't we just use a normal neural network?

The network doesn't inherently have a concept of: "This was the previous input."

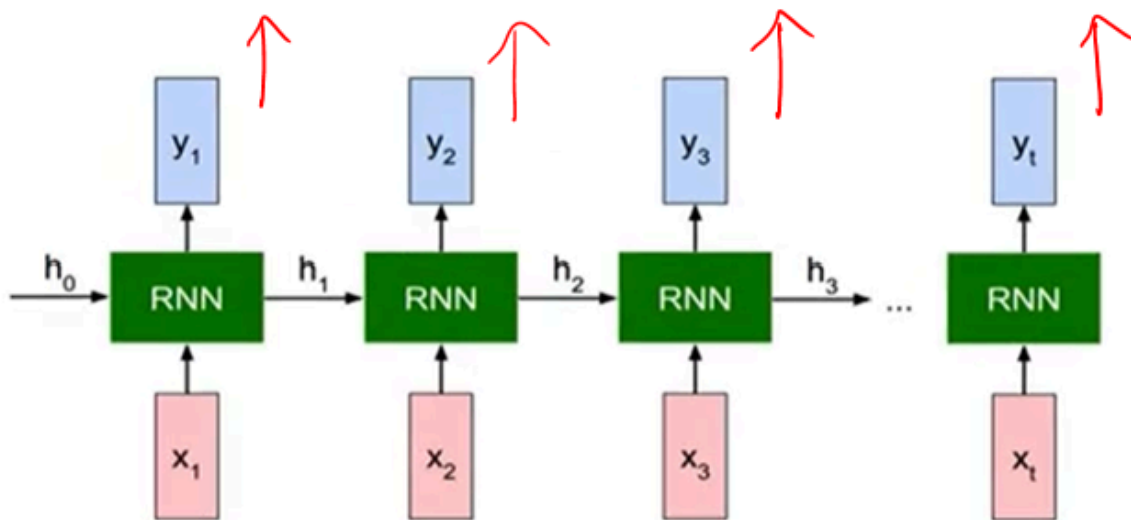
A feed forward neural network is roughly image → cnn → feature vector → classifier → cat

If I give image A and then image B the processes B without automatically remembering A.

Enter RNN's (Recurrent Neural Network) It is designed specifically for sequences.

The important idea is that an RNN maintains a hidden state that acts like a form of memory .

The hidden(internal state) as retaining information from the past data in the same sequence



At every time step , the RNN receives

1. Current input
2. Previous hidden state

And produces

1. Current hidden state
2. output

RNN blocks share same weights $RNN_1 \rightarrow RNN_2 \rightarrow RNN_3 \rightarrow RNN_4$ it might look like four different networks but they are actually the same RNN function reused repeatedly

The four major RNN structures

1. Many to many
2. Many to one
3. One to many
4. One to one

20. Vanilla RNN

Now let's understand the actual mathematics.

The fundamental RNN equation is:

$$h_t = \tanh(Ux_t + Wh_{t-1} + b)$$

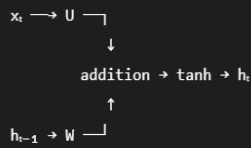
where:

- x_t = current input
- h_{t-1} = previous hidden state
- U = input-to-hidden weights
- W = hidden-to-hidden weights
- b = bias
- h_t = current hidden state

The PPT describes these three weight matrices as U , W , and V .

The equation actually meant current memory = current input + previous memory

More precisely:



So the RNN asks:

What should I remember from the past, and what should I learn from the current input?

Why $\tanh(x)$ is used as activation where outputs values between -1 and 1 it introduces non-linearity. Without an activation function, repeated linear transformations would not give the network enough expressive power.

22. Output equation

After calculating h_t , we can calculate the output:

$$y_t = \text{softmax}(Vh_t + b_y)$$

Here:

- h_t = current hidden state
- V = hidden-to-output weight matrix
- y_t = output probability

Training an RNN Now we reach one of the most important sections (Backpropagation Through Time (BPTT))

Forward pass



Loss



Backpropagation



Gradient



Update weights

1. Forward pass : The RNN calculates h_1, h_2, h_3 and outputs predicts
2. Loss : Then we calculate the standard cross entropy loss. $\text{Loss} = E$
3. Gradient: So we then calculate the gradient, the error has to travel backward hence it is called Backpropagation Through Time. $\text{gradient} = dE/d\text{Weight}$
4. Then we update the weights $U = U - \text{gradient}$, similarly for V, W

31. The Vanishing Gradient Problem

Now we encounter one of the biggest weaknesses of vanilla RNNs.

Suppose we have a very long sequence:

$x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4 \rightarrow \dots \rightarrow x_{100}$



Imagine the information in x_1 is important for predicting y_{100} .

The gradient must travel through many RNN steps:

$100 \rightarrow 99 \rightarrow 98 \rightarrow \dots \rightarrow 1$



During backpropagation, gradients involve repeated multiplication.

If the values being multiplied are smaller than 1:

$$0.5 \times 0.5 \times 0.5 \times \dots$$

the result becomes extremely small.

For example:

$$0.5^{10} = 0.000976$$

and:

$$0.5^{100} \approx 7.9 \times 10^{-31}$$

Essentially zero.

Therefore:

Earlier time steps receive almost no gradient.

This is the **vanishing gradient problem**.

The PPT describes this as earlier time steps losing influence over later outputs.

Why does vanishing gradient matter? To understand the final prediction we need information from much previous information in the sentence. But if the sequence is long, vanilla RNN may effectively forget early information. vanilla RNNs struggle with **long-term dependencies**.

Exploding gradient the opposite can also happen

Suppose gradients repeatedly get multiplied by values greater than 1 : $2 \times 2 \times 2 \times \dots$ then $2^{10} = 1024$, 2^{100} is enormous the gradients become extremely large this is called exploding gradient problem one common solution is : **Gradient clipping** If gradient magnitude becomes too large, we limit it gradient = 1000 clip it gradient = 5

So we change the RNN architecture and that's where LSTM and GRu come in

LSTM (Long Short Term Memory)

The main purpose : Solve the long-term dependency problem of vanilla RNNs.

LSTM as having both a hidden state and an internal cell state that stores long-term information.

Cell state = long-term memory

Hidden state = current working/output information

LSTM has gates the three major gates

1. Forget gate

The forget gate decides: How much of the old cell state should we keep?

Formula:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

The sigmoid function gives:

$$0 \rightarrow 1$$

Interpretation:

0 → forget completely
1 → keep completely

2. Input gate : What new information should we store?

The input gate decides how much new information should enter the cell state.

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Again:

0 → don't store
1 → strongly store

43. Updating the cell state

The new cell state is:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Old cell state → forget gate → important old info → new candidate info → input gate
→ new cell state

3. Output gate : What part of the cell state should I expose as the hidden state/output?

Formula:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

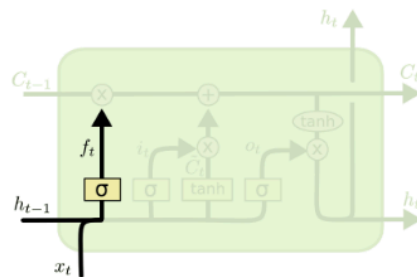
Then:

$$h_t = o_t * \tanh(C_t)$$

Step-by-Step LSTM Walk Through

The first step in our LSTM is to decide what information we're going to throw away from the cell state. This decision is made by a sigmoid layer called the "forget gate layer." It looks at h_{t-1} and x_t , and outputs a number between 0 and 1 for each number in the cell state C_{t-1} . A 1 represents "completely keep this" while a 0 represents "completely get rid of this."

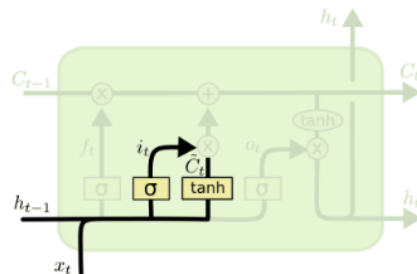
Let's go back to our example of a language model trying to predict the next word based on all the previous ones. In such a problem, the cell state might include the gender of the present subject, so that the correct pronouns can be used. When we see a new subject, we want to forget the gender of the old subject.



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

The next step is to decide what new information we're going to store in the cell state. This has two parts. First, a sigmoid layer called the "input gate layer" decides which values we'll update. Next, a tanh layer creates a vector of new candidate values, $\tilde{C}_t$, that could be added to the state. In the next step, we'll combine these two to create an update to the state.

In the example of our language model, we'd want to add the gender of the new subject to the cell state, to replace the old one we're forgetting.



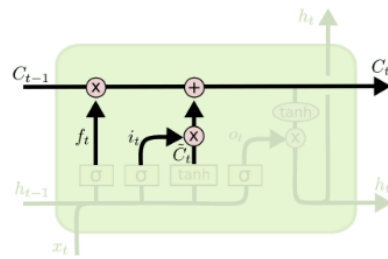
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

It's now time to update the old cell state, C_{t-1} , into the new cell state C_t . The previous steps already decided what to do, we just need to actually do it.

We multiply the old state by f_t , forgetting the things we decided to forget earlier. Then we add $i_t * \tilde{C}_t$. This is the new candidate values, scaled by how much we decided to update each state value.

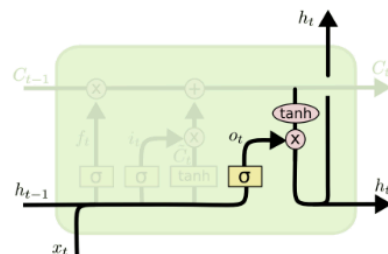
In the case of the language model, this is where we'd actually drop the information about the old subject's gender and add the new information, as we decided in the previous steps.



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Finally, we need to decide what we're going to output. This output will be based on our cell state, but will be a filtered version. First, we run a sigmoid layer which decides what parts of the cell state we're going to output. Then, we put the cell state through $\tanh$ (to push the values to be between -1 and 1) and multiply it by the output of the sigmoid gate, so that we only output the parts we decided to.

For the language model example, since it just saw a subject, it might want to output information relevant to a verb, in case that's what is coming next. For example, it might output whether the subject is singular or plural, so that we know what form a verb should be conjugated into if that's what follows next.



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Why does lstm reduce vanishing gradients ? this is the deeper idea

The cell state provides a relatively direct path for information rather than $h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow h_4$ in rnns. The PPT calls this a **gradient highway**, because information/gradients can flow more effectively across time. That's why LSTMs can preserve long-term information much better than vanilla RNNs.

GRU (Gated recurrent unit)

It is another solution to the problem of vanilla RNNs and GRU is simpler than LSTM

We only have 2 gates

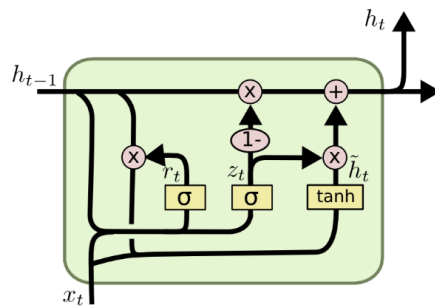
1. Update gate
2. Reset gate

Reset gate : How much should I ignore the previous state when creating new information?

Reset = 0 $\rightarrow$ mostly ignore old info

Reset = 1 $\rightarrow$ use old info

A slightly more dramatic variation on the LSTM is the Gated Recurrent Unit, or GRU, introduced by Cho, et al. (2014). It combines the forget and input gates into a single “update gate.” It also merges the cell state and hidden state, and makes some other changes. The resulting model is simpler than standard LSTM models, and has been growing increasingly popular.



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$